
Policy Iteration Lower Bounds for Multi-Action MDPs

Sahil Shah
160050005

Naman Jain
160050025

Nitish Joshi
160070017

Sriram YV
16D070017

Abstract

In this project, we work on the problem of finding lower bounds for the simple policy iteration algorithm for multi-action MDPs. Finding lower bounds $g(n)$ refers to finding a family of MDPs such that a given policy iteration algorithm takes $g(n)$ steps to converge to the optimal policy of an MDP from the said family having n nodes. Motivated by Melekopoglou and Condon [1994] which prove lower bounds for 2-action MDPs, we design k -action MDPs which obtain strong lower bounds of $O(k2^n)$. We also provide a rough proof sketch to show that simple policy iteration requires exactly $k2^{n-1} - 1$ steps for convergence.

1 Introduction

Markov Decision Processes (MDPs) [Puterman, 1994] are mathematical models for sequential decision making which are widely used in diverse fields ranging from queuing systems to reinforcement learning. Formally, an MDP M is defined to be $M = (S, A, R, T, \gamma)$ where S are the set of states which the agent can be in and A are the set of actions which the agent can take from any state. T guides the transitions from one state to another - $T(s, a, s')$ is the probability of transitioning from state s to s' if action a was taken from s and $R(s, a, s')$ is the reward obtained in the process. $\gamma \in [0, 1)$ is the discounting factor which helps control the importance of future rewards.

A policy π is defined to be a mapping from S to A : $\pi(s)$ is the action which the agent takes when following π . The objective in an MDP then is to find a policy which obtains maximum expected cumulative reward. To understand this concretely, a value function for a policy π is defined on the states S as follows:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (1)$$

where s_0 is the starting state of MDP and r_t is the reward obtained at time step t . It was shown in Bellman [1957] that if Π is the set of all policies for M , then there exists an optimal policy π^* such that $\forall s \in S$ and $\forall \pi \in \Pi$ we have,

$$V^{\pi^*}(s) \geq V^\pi(s) \quad (2)$$

The policy iteration family of algorithms [Howard, 1960] exploit (2) to find improvable states at each step of the algorithm and switch the action taken at that state to obtain a new policy. In this work following Melekopoglou and Condon [1994], we analyse the **Simple Policy Iteration Algorithm** where only one of the improvable states is switched at each time step. We use the convention of choosing the switchable state with the highest index.

To summarize, the main contributions and findings of the work are as follows:

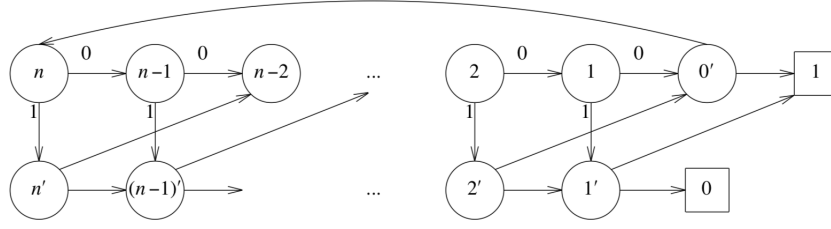


Figure 1: The family of MDPs considered in Melekopoglou and Condon [1994]

1. Propose a family of n -node, k -action MDPs which require exactly $k2^{n-1} - 1$ number of steps for convergence in simple policy iteration. We provide extensive empirical evaluation as well as discussion on these bounds and some intuitions for a proof sketch.
2. We briefly touch on a few other families that provide weaker bounds like $O(k2^{n/k})$ which we came across during our experiments.
3. Implemented a framework that allows us to quickly and easily define family of MDPs similar in structure to those defined in Melekopoglou and Condon [1994] to empirically determine the dependence on the number of actions and nodes.

2 Background

In this section we provide some notations and definitions that will be used in the construction of our MDPs. We also describe the 2-action MDP family which was proposed in Melekopoglou and Condon [1994] which provides a lower bound of $2^n - 1$ iterations.

Decision nodes are defined to be nodes with k actions each having a deterministic transition to a next state. Average nodes have k identical actions with equal probability of transitioning to any of its successor states. In all our MDPs we consider average nodes having only 2 successors (as in Melekopoglou and Condon [1994]). Finally, sink nodes are defined to be terminal nodes (no outgoing transitions) with a fixed value assigned to them.

Melekopoglou and Condon [1994] consider the problem of *minimizing* the value of every node in an MDP which consists of decision nodes, average nodes and sink nodes. We consider n , the number of nodes to be the number of decision nodes since average nodes and sink nodes can be made part of the environment as well. In the graphical representation of the MDP, each arrow from a decision node corresponds to a deterministic choice of node that the agent will find itself at if it chooses that action. Melekopoglou and Condon [1994] consider decision nodes with only 2 successors, whereas our decision nodes have k successors. Decision nodes are indexed with the number set $1, 2, \dots, n-1, n$ in Fig. 1, and depicted with blue nodes in Fig. 2.

Average nodes are indexed with the set $1', 2', \dots, (n-1)', n'$ in Fig. 1 and represented with purple nodes in Fig. 2. Melekopoglou and Condon [1994] considers only 2 sink nodes: with values 0 and 1, whereas we consider sink nodes with arbitrary values. Sink nodes are always represented with square nodes in both the figures.

We say that a node is *improvable* if there is an action such that the action-value function for that node-action pair as per the current policy is better than the state-value function of that node under the current policy. Any such action is said to be an *improvable* action at that node.

The family of MDPs that they consider is depicted in Fig. 1. The **Simple Policy Iteration Algorithm** deterministically chooses the node with the largest label from the set of improvable nodes. When $k > 2$, we also follow the convention of choosing the action with highest index when choosing which action to switch to for current node. Note that this issue of choosing which action to switch to does not arrive for $k = 2$. For the family shown in Fig. 1, the optimal policy is taking action 1 at node 1 and action 0 at all other nodes. However, if one starts with the policy of taking action 0 at all nodes, it takes $2^n - 1$ iterations to converge to the optimal policy. We leave out the details of the proof and encourage readers to look at Melekopoglou and Condon [1994] for details.

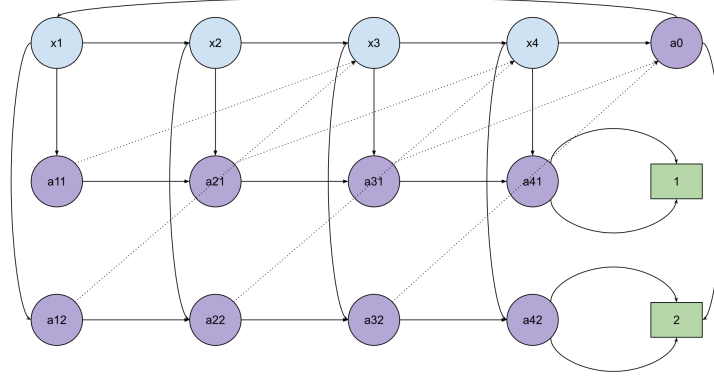


Figure 2: Illustration of our MDP for $n = 4$ and $k = 3$. The nodes in blue are the decision nodes, the nodes in purple are the average nodes and the green nodes are the sink nodes.

3 The $O(k2^n)$ Family of MDPs

In this section we discuss our strongest and primary result: a family of MDPs with k actions and n nodes such that simple policy iteration takes $k2^{n-1} - 1$ steps to converge.

3.1 Primary Ideas

In the family of MDPs introduced in Melekopoglou and Condon [1994] which is shown in Fig. 1, there is a layer of decision nodes under which there is a layer of average nodes. For a k action MDP, we introduce a total of $k - 1$ layers of average nodes and index them in the following manner: nodes x_i for $1 \leq i \leq n$ refer to the top level decision nodes. Nodes $a_{i,j}$ for $1 \leq i \leq n$ and $1 \leq j \leq k$ refer to the average node in the j^{th} layer under the i^{th} decision node. There is an additional, special average node that is the result of taking action 0 from the rightmost decision node in the original MDP in Fig. 2, which we denote by a_0 . Let $Action(x_i, l)$ denote the deterministic node that the agent will find itself in on taking action l , $0 \leq l \leq k$ from decision node x_i . Then we define the $Action(x_i, l)$ function as:

$$Action(x_i, l) = \begin{cases} x_{i+1} & i < n, l = 0 \\ a_0 & i = n, l = 0 \\ a_{i,l} & \text{otherwise} \end{cases}$$

Essentially, each decision node encodes the decision of going to the next decision node, or to any of the average nodes in the column under it. We now define the value at the average nodes as follows:

$$V(a_{i,j}) = \begin{cases} \frac{V(a_{i+1,j}) + V(x_{i+2})}{2} & i < n - 2 \\ \frac{a_0 + V(a_{i+1,j})}{2} & i = n - 1 \\ (c(n, k))^j & \text{otherwise} \end{cases}$$

$$V(a_0) = \frac{V(x_1) + (c(n, k))^k}{2},$$

where $c(n, k)$ is a constant that varies with the number of states and actions in the MDPs. $c(n, k)^j$ is essentially the value of the j th sink node.

An instance of the MDP for the case $n = 4$ and $k = 3$ is shown in the Fig. 2. The MDP that we have constructed, at a high level, has $k - 1$ copies of the average layer of the original MDP, with each

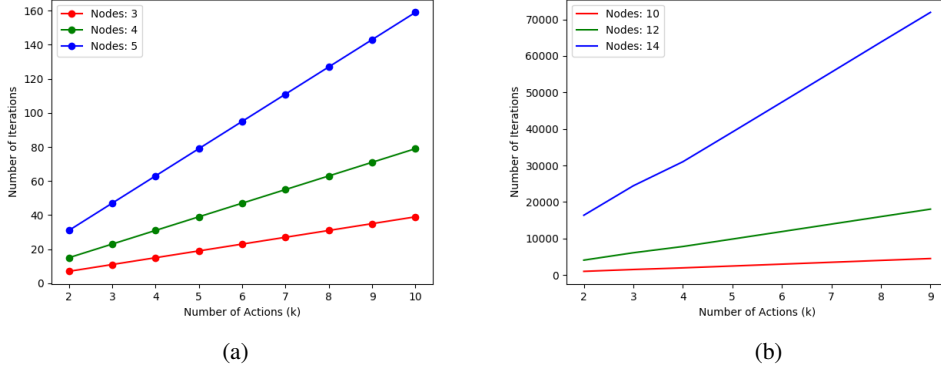


Figure 3: Graph showing number of iterations for convergence of policy iterations compared to the number of nodes. We plot up to $k = 10$ for $n = 3, 4, 5$ using $c(n, k) = 2^n$ in the left subfigure. We use an approximation to $c(n, k) = 2^n$ which fits in float64 for $n = 10, 12, 14$ in the right subfigure to get an approximate graph.

layer differing only in the value of the rightmost average node, which increases as we move down. The rate of increase is a function of the size of the MDP. Recall that in our formulation, we hope to minimize the value of each decision node and so the optimal policy is action 1 for the rightmost decision node and action 0 at all other nodes.

We now describe the intuition behind such a construction. Melekopoglou and Condon [1994] provides an MDP in which the actions chosen oscillate for the larger-numbered nodes for each action choice of the smaller-numbered nodes, if one starts with the policy of taking action 0 at every decision node. We hope to change this oscillation between 2 actions to a cycling through each of the actions. Now, since our policy iteration algorithm selects the improvable action with the largest index, we wish to create a situation in which the current action taken at a node is 0, the optimal action (after fixing the actions taken at all other nodes) is 1, but actions 2, 3, ..., $k - 1$ are also improvable. Since the algorithm tries to improve nodes with the largest index first, the algorithm will consider each action in decreasing order and finally settle on action 1 before moving onto the next node. Now, in order to ensure that actions 1, 2, 3, ..., $k - 2, k - 1$ are increasingly worse choices we increase the value of the rightmost average node in a layer. Since all the other connections are symmetric across layers, the value of node $a_{i,j}$ will vary with the value of $a_{n,j}$ since the successive averaging across nodes will add a term of $2^{-(n-i)}V(a_{i,n})$ to $V(a_{i,j})$. Hence, layers with lesser values of $V(a_{n,j})$ will be better choices and since we choose $V(a_{n,j}) = (c(n, k))^j$, we ensure that actions $k - 1, k - 2, \dots, 2, 1$ are progressively better choices. Now, we also need to ensure that each of these actions is a better choice than action 0, and this will be true if we choose $c(n, k)$ to be large enough.

As a result, each iteration which changes the action for a node from 0 to 1 in the original MDP is converted to $k - 1$ iterations ($0 \rightarrow k - 1 \rightarrow \dots \rightarrow 2 \rightarrow 1$) for the MDP we defined. In the family defined in Melekopoglou and Condon [1994], there are a total of $2^n - 1$ iterations for an MDP with n nodes, and of these $2^{n-1} - 1$ correspond to iterations which switch the action from 0 to 1, and the remaining $2^{n-1} - 1$ correspond to iterations switching the optimal action from 1 to 0. Since each of the first 2^{n-1} iterations now take $k - 1$ iterations each, we have a total of $(k - 1)2^{n-1} + 2^{n-1} - 1 = k2^{n-1} - 1$. Clearly, the family we are describing is a generalization of the family provided in Melekopoglou and Condon [1994], and for $k = 2$, it degenerates into that family. These ideas can be converted into a rough proof sketch provided in Section 3.3

3.2 Empirical Results

We used the framework we built (described in 4.1) to construct the family of MDPs as defined above and verify the number of iterations taken for simple policy iteration to converge. The only issue that warrants a discussion in this process is the choice of $c(n, k)$ and the implementation issues that arise due to this choice. We find that for every instance of family of MDPs, there is a value that leads to

exactly $k2^{n-1} - 1$ iterations and we found that setting $c(n, k) = 2^n$ always worked, but for large values of n and k , $V(a_{n,j}) = 2^{nj}$ becomes very large for large j . This causes numeric instability which causes the equality of large, close-by values to evaluate to true as a result of which the policy improvement algorithm starts oscillating between two assignments of actions to the nodes.

Due to these issues, we reduce the value of $C(n, k)$ for instances with a large number of nodes which in turn causes the number of iterations to fall slightly below $k2^{n-1} - 1$ because for some action 0 to action 1 transitions, the value associated with choosing higher index actions (close to $k - 1$) exceeds the value associated with action 0 at some decision nodes. As discussed in Section 3.1, choosing $V_{n,j} = (c(n, k))^j$ ensures that actions $k - 1, k - 2, \dots, 2, 1$ are progressively better choices, but the actual value of $c(n, k)$ determines whether or not each of these is better than action 0. Finally, choosing smaller values of $c(n, k)$ lead to a slightly reduced number of iterations as seen in the plot in Fig. 3b. Future work could focus on an implementation of the simple policy iteration that focuses on numerical stability and uses high precision arithmetic operations.

We also show in Fig 3a the dependence of the number of iterations on k for different values of n and find that for smaller values (2^{nk} small enough to fit in float64) of n and k , the number of iterations exactly satisfies the formula $k2^{n-1} - 1$.

3.3 Proof Sketch

We describe a rough proof sketch of the MDPs proposed in Melekopoglou and Condon [1994] and how that can be extended for our MDPs.

Firstly observe that similar to Melekopoglou and Condon [1994], if vertex k is switchable and any of the larger numbered vertices are switched, then vertex k is still switchable. To understand why this should follow in our setting, note that there are no connections between nodes of two different average layers, so the values of each average node only depend on that layer and the nodes in the min layer. Thus each pair of (min nodes, avg nodes) can be thought of as separate instance of the MDP in Melekopoglou and Condon [1994] and a similar proof should follow (Indeed this will require a more formal analysis).

The second thing to observe is that at any point of the policy iteration algorithm, there can be at most one node whose action is between 2 to $k - 1$. Note that for a node to have one of these actions for the first time, that node must have been switched at that point in the algorithm since we start with all actions 0. Hence that node is still switchable because each of these actions is always switchable. Thus no higher numbered node can have that action since otherwise that node will be switched first.

Next note that a result similar to Lemma 2.6 of Melekopoglou and Condon [1994] which states that if vertex p is switched and $S_1 \dots S_{p-1} = 0 \dots 01$ then all vertices $1, 2, \dots, p - 1$ are switchable should follow in our case. This is because the node p will be switched from action $k - 1$ to action 1 as described in the above paragraph - this reduces our MDP to having one min layer and one average layer similar to the original MDP in Melekopoglou and Condon [1994]. (Again we leave out the finer details which require exactly characterizing when a node is switchable for future work.)

Analogous to Lemma 2.7 of Melekopoglou and Condon [1994], we state and prove the following lemma:

Lemma 3.1 *The following two statements hold (for every positive p):*

- *If $S_1 \dots S_{p-1} S_p = 0 \dots 01$, and the nodes $x_1, x_2, \dots, x_p$ are improvable, the next $k2^{p-1} - 1$ iterations of the difference policy improvement algorithm are made on these vertices, to reach the policy where $S_1 \dots S_{p-1} S_p = 0 \dots 00$.*
- *If $S_1 \dots S_{p-1} S_p = 0 \dots 00$, and the nodes $x_1, x_2, \dots, x_p$ are improvable, the next $k2^{p-1} - k + 1$ iterations of the difference policy improvement algorithm are made on these vertices, to reach the policy where $S_1 \dots S_{p-1} S_p = 0 \dots 01$.*

We prove this lemma by induction on p . As a building block of the proof we use the fact that switching from action 1 to action 0 at a node takes only 1 iteration of the policy improvement algorithm and this statement follows from the preceding arguments assuming that the node in question is switchable. In order to switch from action 0 to action 1, we assume that we first switch to action $k - 1$, then to action $k - 2$ and so on until we reach action 1. As stated before, due to the choice of $V(a_{n,j}) = (c(n, k))^j$

actions $k - 1, k - 2, \dots, 2, 1$ are increasingly better choices. Since the node is improvable, action 1 is better than action 0, and for a large enough $c(n, k)$ we find that each of actions $k - 1, \dots, 2, 1$ is better than action 0 and so switching from action 0 to action 1 for this node will take $k - 1$ iterations. However, in order to find a satisfactory value of $c(n, k)$ requires a rigorous analysis to find a closed form expression for the value of any node x_i or $a_{i,j}$ as a function of i, j and the policy $S_1 S_2 \dots S_n$. This could form part of future work, and for now we assume that switching from action 0 to action 1 is guaranteed to take $k - 1$ iterations.

Base Case: For $k = 0$, the statement follows directly from the arguments and assumptions in the previous paragraph.

Induction Hypothesis: Assuming $x_1, x_2, \dots, x_p$ are improvable, going from $S_1 \dots S_{p-1} S_p = 0 \dots 01$ to $S_1 \dots S_{p-1} S_p = 0 \dots 00$ takes $k2^{p-1} - 1$ iterations, and going from $S_1 \dots S_{p-1} S_p = 0 \dots 00$ to $S_1 \dots S_{p-1} S_p = 0 \dots 01$ takes $k2^p - k + 1$ iterations.

Induction Step: Assuming $x_1, x_2, \dots, x_p, x_{p+1}$ are improvable, going from $S_1 \dots S_p S_{p+1} = 000 \dots 001 \rightarrow 000 \dots 000$ can be considered as first going to $S_1 \dots S_p S_{p+1} = 000 \dots 011$, which by the induction hypothesis takes $k2^{p-1} - 1$ steps. Next, switching S_{p+1} from 1 to 0 to get to $S_1 \dots S_p S_{p+1} = 000 \dots 010$ takes 1 step. Finally, by the induction hypothesis the move $S_1 \dots S_p S_{p+1} = 000 \dots 010 \rightarrow 000 \dots 000$ takes $k2^{p-1} - k + 1$. Hence, a total of $k2^p - k + 1$ steps are taken, which completes the proof for this case.

The other case can be proved analogously, with $S_1 \dots S_p S_{p+1} = 000 \dots 000 \rightarrow 000 \dots 010$ taking $k2^{p-1} - 1$ iterations (by the induction hypothesis, switching S_{p+1} from 0 to 1 taking $k - 1$ iterations and finally $S_1 \dots S_p S_{p+1} = 000 \dots 011 \rightarrow 000 \dots 001$ taking $k2^{p-1} - k + 1$ iterations. The total number of iterations is hence $k2^p - 1$

Finally, using the result analogous to Lemma 2.6 of Melekopoglou and Condon [1994] as described above, and from Lemma 3.1, it follows that switching from the starting policy of $S_1 S_2 \dots S_n = 000 \dots 000$ to the optimal policy $S_1 S_2 \dots S_n = 000 \dots 001$ takes $k2^{n-1} - 1$ iterations.

4 Other Details

In this section we will discuss the framework we built to quickly and conveniently experiment with different families and the experiments performed on different MDP families.

4.1 The Code Framework

In order to aid with our experiments, we coded up an easy-to-use framework that abstracts out solving the MDP and defining a family of MDPs. Hence, for each family we need to define a function which takes as input the number of state (n) and the number of actions (k) and produces an encoding of the MDP which serves as input to a function that implements the policy iteration. An MDP is defined as a graph with decision, average and sink nodes as defined in Section 2, where decision nodes can have a variable number of successors depending on the number of actions at that node. The policy improvement algorithm uses matrix inversion to solve Bellman's equations in every iteration and then identifies improvable nodes and actions using a pre-determined protocol.

We can easily define the details of the policy improvement algorithm as well as experiment with different families to ascertain their dependence on n and k . We plot graphs of the number of iterations takes as a function of k for different values of n in order to determine the dependence on n and k . We observe the variations in the policy over iterations to gain insights and update the family of MDPs.

4.2 The $O(k2^{n/k})$ family of MDPs

For our initial experiments we used an intuitive design for including factor of k in our lower bound. We simply replicate the mdp from Melekopoglou and Condon [1994] k times and then add a new start state with k actions such that i^{th} action leads it to the original start state of original i^{th} layer mdp. Therefore the MDP has n decision nodes in each of the k layers giving a total of $n' = n * k + 1$ decision nodes. We expected this MDP to require $k2^{n'/k}$ iteration for convergence which we verify in the graphs in 4.

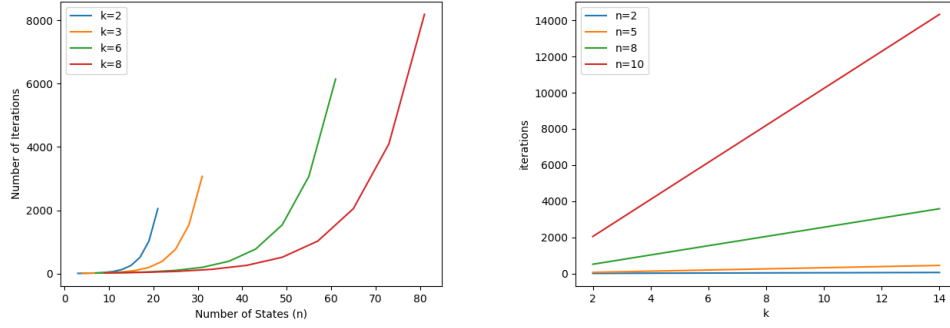


Figure 4: Graphs showing number of iterations for family of mdps with lower bounds as $O(k2^{n/k})$. Figure (a) shows number of iterations increasing with number of states (n) for four different number of actions (k). Figure (b) shows a linear increase in number of iterations with number of actions (k) with number of states per layer held constant.

5 Conclusion

In this work we provided a family of MDPs in which the minimum number of iterations needed for convergence under a Simple Policy Iteration Algorithm is $O(k2^{n/k})$. This result is novel since previous work considered MDPs with 2 actions only, and hence did not provide any understanding of the dependence of the lower bound on the maximum number of actions. Besides extensive empirical experimentation, we have also provided a sketch of the proof which future work can package into a complete proof by performing some in-depth analysis to obtain closed form expressions for the state value function as a function of the policy, as done in the proof for 2-action MDPs.

References

- Mary Melekopoglou and Anne Condon. On the complexity of the policy improvement algorithm for markov decision processes. *INFORMS Journal on Computing*, 6:188–192, 1994.
- Martin L. Puterman. Markov decision processes. *Wiley*, 1994.
- Richard Bellman. Dynamic programming. *Princeton University Press*, 1957.
- Ronald A. Howard. Dynamic programming and markov processes. *MIT Press*, 1960.